Trabalho Prático: Divisão e Conquista

[Nome 1] e [Nome 2] · Novembro de 2026

1. Introdução

Este relatório descreve a implementação, em TypeScript, de dois problemas resolvidos por divisão e conquista (D&C), o subarranjo de soma máxima e o par de pontos mais próximos, ambos comparados a uma solução de força bruta. O objetivo é verificar se o tempo medido segue a complexidade prevista pela recorrência de cada algoritmo. O código, os casos de teste e as medições brutas estão no repositório: [link do repositório].

Premissas adotadas

O enunciado deixa alguns pontos em aberto. As decisões tomadas foram:

- **Índices:** a saída numera as posições a partir de 0.
- **Empates na Etapa 1:** vale o menor início e, depois, o menor fim. As quatro versões devolvem exatamente o mesmo intervalo.
- **Empates na Etapa 2:** qualquer par com a distância mínima é aceito.
- **Formato da Etapa 2:** a entrada é n seguido de n pares x y; a saída é a distância e os índices das duas estações.
- **Versão O(n):** a "questão desafio" citada nos critérios não é definida no enunciado. Foi interpretada como uma versão linear da Etapa 1, implementada de duas formas: o algoritmo de Kadane e uma D&C com combinação em tempo constante.
- **Limiar:** foi interpretado como o tamanho em que a recursão da Etapa 2 passa a usar força bruta.
- **Medição de tamanhos:** a Etapa 3 fala em "dígitos" para a Etapa 2; os valores foram tratados como número de pontos.

2. Etapa 1: subarranjo de soma máxima

Dado um vetor de n inteiros, procura-se o intervalo contínuo de maior soma.

Força bruta. Para cada início, percorre todos os fins possíveis, reaproveitando a soma parcial. O número de intervalos é $n(n + 1)/2$, logo $T(n) = \Theta(n^2)$.

Divisão e conquista. O vetor é dividido ao meio, e a resposta é a maior entre a melhor da esquerda, a melhor da direita e a melhor que cruza o meio. A última sai de duas varreduras a partir do meio, em $\Theta(n)$:

$$T(1) = \Theta(1) \qquad T(n) = 2T(n/2) + \Theta(n)$$

Pelo Teorema Mestre, com $a = 2$, $b = 2$ e $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$, vale o caso 2, e $T(n) = \Theta(n \log n)$. A árvore de recursão mostra o mesmo: são $\log_2 n$ níveis, cada um com custo total $\Theta(n)$.

D&C com combinação constante. Cada chamada devolve quatro valores do seu trecho: a soma total, o melhor prefixo, o melhor sufixo e o melhor intervalo. Com eles, as duas metades são combinadas sem varrer o vetor:

$$T(1) = \Theta(1) \qquad T(n) = 2T(n/2) + \Theta(1)$$

Como $f(n) = \Theta(1) = O(n^{1-\varepsilon})$, vale o caso 1 do Teorema Mestre, e $T(n) = \Theta(n)$.

Kadane. Não usa divisão e conquista: guarda a melhor soma que termina em cada posição e recomeça somente quando a soma acumulada fica negativa. Assim, $T(n) = T(n - 1) + \Theta(1) = \Theta(n)$.

3. Etapa 2: par de pontos mais próximos

Força bruta. Compara os $n(n - 1)/2$ pares, logo $T(n) = \Theta(n^2)$. As comparações usam a distância ao quadrado, e a raiz só é calculada na resposta.

Divisão e conquista. Os pontos são ordenados por x e por y uma única vez, com a função da biblioteca padrão, em Θ(n log n). A recursão divide na mediana de x e preserva a ordem por y ao separar a lista de cada lado, em Θ(n), sem ordenar de novo. Após resolver os dois lados e obter δ, percorre a faixa de largura 2δ em ordem de y. Abaixo do limiar L, usa força bruta:

$$T(n) = \Theta(1) \text{ para } n \le L \qquad T(n) = 2T(n/2) + \Theta(n) \text{ para } n > L$$

Vale novamente o caso 2 do Teorema Mestre, com total Θ(n log n), somado à ordenação inicial, de mesma ordem. Para L fixo, o custo detalhado é $\Theta(n \log(n/L) + nL)$: há cerca de $\log_2(n/L)$ níveis de combinação e cerca de n/L folhas de custo $\Theta(L^2)$. O primeiro termo diminui com L e o segundo cresce, o que motiva a busca por um limiar intermediário (seção 6.3).

Por que bastam 7 comparações por ponto

Seja δ a menor distância encontrada nos dois lados. Um par melhor, com um ponto em cada lado, precisa estar na faixa e ter diferença de y menor que δ. Fixado um ponto p da faixa, os candidatos seguintes em ordem de y estão em um retângulo de largura 2δ e altura δ, centrado na linha divisória.

Divida o retângulo em 8 quadrados de lado δ/2, 4 em cada lado da linha. Dois pontos do mesmo lado têm distância de pelo menos δ, mas dois pontos no mesmo quadrado estariam a no máximo $\delta/\sqrt{2} < \delta$. Portanto, cada quadrado guarda no máximo um ponto do seu lado, e o retângulo guarda no máximo 8 pontos. Como p é um deles, restam no máximo 7 candidatos. Isso limita a combinação a Θ(n), e a implementação ainda interrompe a busca assim que a diferença de y alcança δ.

4. Testes

A bateria é executada por `npm test`. Ela tem 26 casos com entrada, saída esperada e justificativa (15 da Etapa 1 e 11 da Etapa 2), passados pelos programas reais com a entrada padrão em cada versão, o que resulta em 82 verificações. Depois, compara a técnica com a força bruta em 3.000 entradas aleatórias por etapa, em 27.000 comparações. A Etapa 1 usa valores em faixa pequena para forçar empates; a Etapa 2 usa grades que forçam duplicatas e x repetidos e testa vários limiares. Todas as 27.082 verificações foram aprovadas.

Etapa	Situação	Por que está na bateria
1	Somente negativos	A resposta é o maior valor isolado, nunca um intervalo vazio de soma 0.
1	Tamanho 1 (positivo, negativo e zero)	Exercita o caso base da recursão.
1	Empates (disjuntos, entre negativos, com prefixo ou sufixo de soma 0, todos zeros)	Revelam diferenças entre as versões; pegam recomeço indevido quando a soma parcial é 0.
1	Resposta à esquerda, à direita e cruzando o meio	Cobrem os três candidatos da combinação.
1	Soma acima de 2^{31}	Revelaria somas limitadas a 32 bits.
2	Dois pontos e um único ponto	Menor entrada válida e entrada sem par, que deve gerar erro.
2	Pontos duplicados	A distância 0 esvazia a faixa.
2	Todos com o mesmo x ou o mesmo y	A divisão e a ordenação por y ficam totalmente empatadas.
2	Par em lados opostos da linha	Só a verificação da faixa encontra a resposta.
2	Quadrado e grades 3×3 e 6×6	Geram empates e o arranjo denso que justifica as 7 comparações.
2	Coordenadas negativas e decimais	Confirmam a ordenação numérica e a entrada não inteira.

Para confirmar que a bateria detecta erros, cinco defeitos foram introduzidos de propósito, um por vez: a faixa com 1 vizinho, a retirada da faixa, o recomeço do Kadane com soma zero, o desempate errado no cruzamento e a D&C linear sem o cruzamento. Todos foram detectados.

5. Metodologia experimental

As medições usaram um processador 13th Gen Intel(R) Core(TM) i5-1335U, com 8 GB de memória, Windows 11 e Node.js 24.14.1 (V8), na tomada e com outros programas abertos. Os tamanhos vão de 100 a 25.600, indo além dos sugeridos no enunciado. Cada tamanho teve 20 execuções, com entradas aleatórias de semente fixa, iguais para todos os algoritmos. Antes de medir, o código é executado várias vezes, e esse tempo é descartado, para que o compilador JIT do V8 otimize o código. A Etapa 1 usa inteiros em $[-1.000, 1.000]$, e a Etapa 2, pontos uniformes em um plano de lado 10^6.

Ameaça à validade corrigida. A primeira versão forçava a coleta de lixo antes de cada medição, o que criou tempos em dois patamares: entradas iguais levavam de 5 a 16 ms, contra cerca de 4 ms sem a coleta forçada. Por isso, ela foi removida, e o custo normal de memória faz parte do tempo medido. Os tempos muito pequenos, abaixo de 0,1 ms, seguem mais sujeitos a ruído e ao JIT.

6. Resultados e discussão

6.1. Crescimento observado e previsto

A Tabela 1 traz a média e o desvio padrão. A última linha estima o expoente k de $T(n) \approx c \cdot n^k$ entre n = 1.600 e n = 25.600: o previsto é 2 para a força bruta, pouco acima de 1 para $\Theta(n \log n)$ e 1 para $\Theta(n)$.

	Etapa 1 (ms)				Etapa 2 (ms)	
n	**Força bruta**	**D&C**	**D&C O(n)**	**Kadane**	**Força bruta**	**D&C**
100	0,0112 ± 0,0049	0,0111 ± 0,0123	0,0148 ± 0,0115	0,0006 ± 0,0001	0,0152 ± 0,0119	0,0533 ± 0,0128
200	0,0225 ± 0,0028	0,0114 ± 0,0047	0,0245 ± 0,0340	0,0008 ± 0,0006	0,0318 ± 0,0133	0,0782 ± 0,0040
400	0,111 ± 0,058	0,0190 ± 0,0060	0,0318 ± 0,0448	0,0010 ± 0,0003	0,0756 ± 0,0401	0,181 ± 0,052
800	0,242 ± 0,009	0,0313 ± 0,0021	0,0289 ± 0,0206	0,0013 ± 0,0002	0,295 ± 0,011	0,420 ± 0,171
1.600	0,963 ± 0,041	0,0695 ± 0,0227	0,0545 ± 0,0326	0,0022 ± 0,0002	1,25 ± 0,09	0,799 ± 0,103
3.200	3,85 ± 0,19	0,140 ± 0,020	0,129 ± 0,050	0,0039 ± 0,0006	5,22 ± 0,20	1,80 ± 0,17
6.400	14,83 ± 0,15	0,304 ± 0,056	0,292 ± 0,064	0,0074 ± 0,0010	20,78 ± 0,88	3,88 ± 0,40
12.800	59,93 ± 0,68	0,648 ± 0,128	0,591 ± 0,076	0,0133 ± 0,0015	85,66 ± 3,26	8,16 ± 0,53
25.600	241,4 ± 5,6	1,29 ± 0,19	1,22 ± 0,67	0,0258 ± 0,0046	409,7 ± 36,7	18,91 ± 2,08
Expoente	**1,99**	**1,05**	**1,12**	**0,89**	**2,09**	**1,14**

Tabela 1. Tempo médio ± desvio padrão, em ms, com 20 execuções por tamanho.

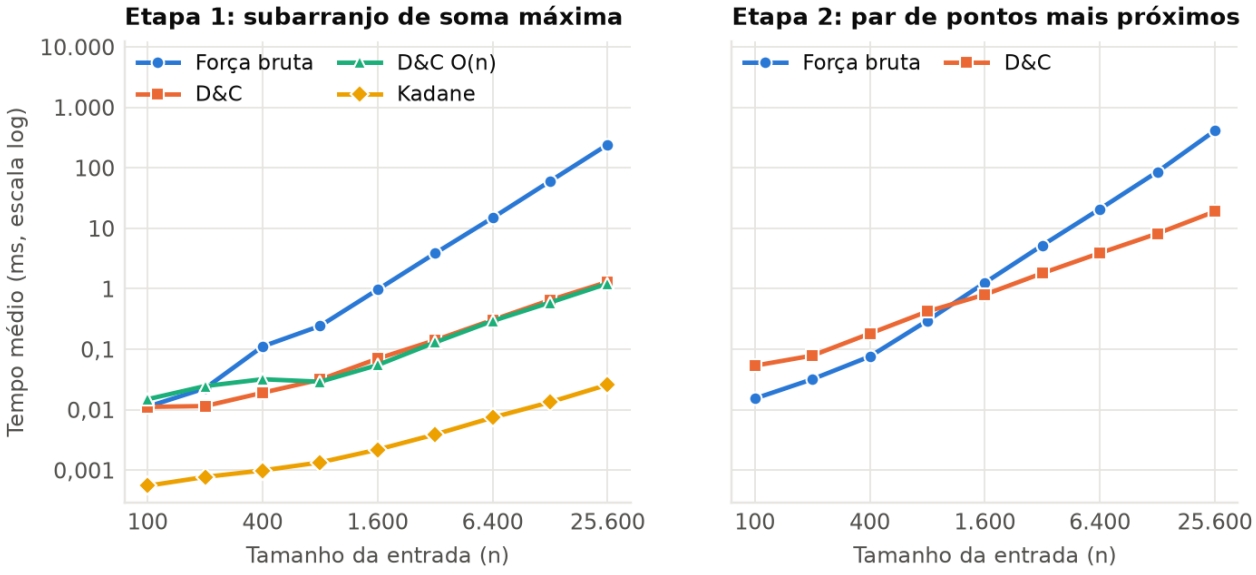


Figura 1. Tempo médio por tamanho de entrada, em escalas logarítmicas.

Na Figura 1, as retas confirmam o previsto. Em n = 25.600, a força bruta da Etapa 1 levou 241,4 ms, contra 1,29 ms da D&C e 0,026 ms do Kadane. Na Etapa 2, foram 409,7 ms contra 18,9 ms. Os expoentes da força bruta foram 1,99 e 2,09, próximos de 2. O valor da Etapa 2 vem principalmente do último salto, de n = 12.800 para 25.600, compatível com efeitos de cache, que não foram medidos diretamente.

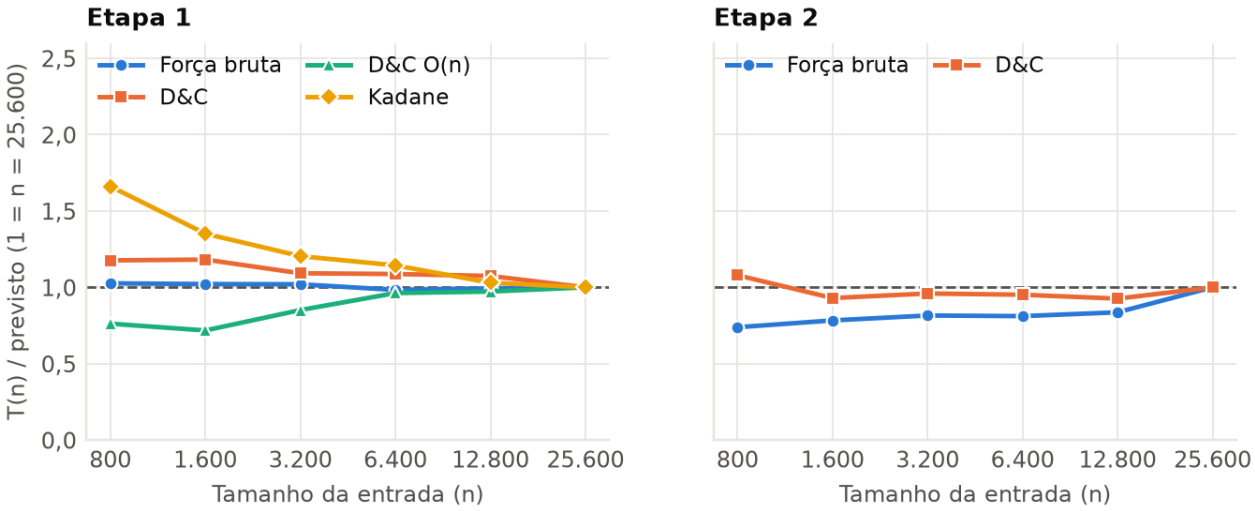


Figura 2. Tempo dividido pelo crescimento previsto (n^2, n log n ou n), com 1 em n = 25.600. Uma linha plana indica concordância com a análise.

A Figura 2 divide cada tempo pelo crescimento previsto; uma curva plana concorda com a análise. A força bruta e a D&C da Etapa 1 são as mais estáveis. As curvas de Θ(n) variam mais, pois seus tempos são muito curtos; isso explica o expoente 0,89 do Kadane, abaixo do valor teórico de 1. Há uma anomalia: em n = 6.400, a D&C O(n) (0,292 ms) quase empata com a D&C Θ(n log n) (0,304 ms), apesar de ter complexidade menor. A D&C O(n) cria um objeto por chamada, e seu custo depende da memória e do JIT, de modo que a ordem assintótica só aparece com entradas maiores. Em n = 25.600, ela já é mais rápida (1,22 ms contra 1,29 ms). O Kadane, sem recursão nem alocação, é o mais rápido em todos os tamanhos.

6.2. Quando a força bruta vence e quando a técnica vence

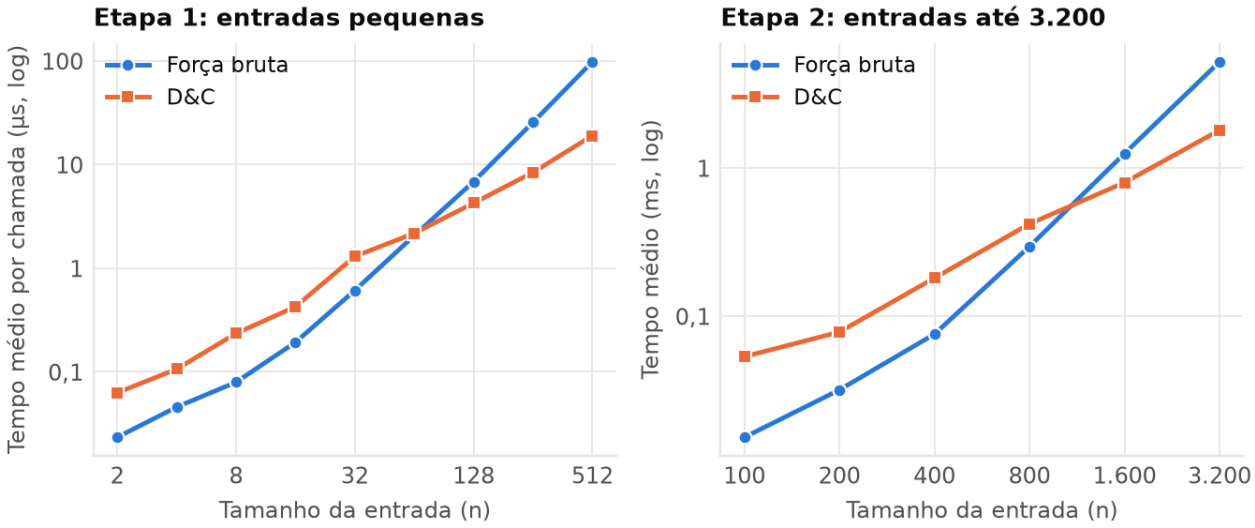


Figura 3. Comparação perto do ponto de virada. Na Etapa 1, cada medição é um lote de chamadas.

Para n pequeno, a constante pesa mais que a ordem de crescimento. A força bruta tem laços simples, sem chamadas nem alocações. Na Etapa 1, ela vence até n = 64, e a D&C passa a vencer em n = 128. Como a D&C gasta pouco em cada chamada, a virada chega cedo.

Na Etapa 2, a virada é bem mais tardia: a força bruta vence até n = 800, e a D&C vence a partir de n = 1.600. O motivo é que a D&C paga duas ordenações com comparador e aloca listas a cada nível. As ordenações, sozinhas, consomem 57% do tempo total em n = 1.600 e 70% em n = 25.600. Depois da virada, a diferença entre as ordens de crescimento domina: ao dobrar n, o tempo da força bruta fica cerca de 4 vezes maior, e o da D&C, pouco mais de 2 vezes.

6.3. Escolha do limiar da Etapa 2

Limiar	2	4	8	16	32	64	128	256
n = 6.400	4,23	4,03	3,86	3,63	3,78	3,54	3,87	4,31
n = 25.600	20,07	19,20	18,29	17,51	17,65	17,79	17,88	19,82

Tabela 2. Tempo médio, em ms, da D&C da Etapa 2 para alguns limiares, com 20 execuções.

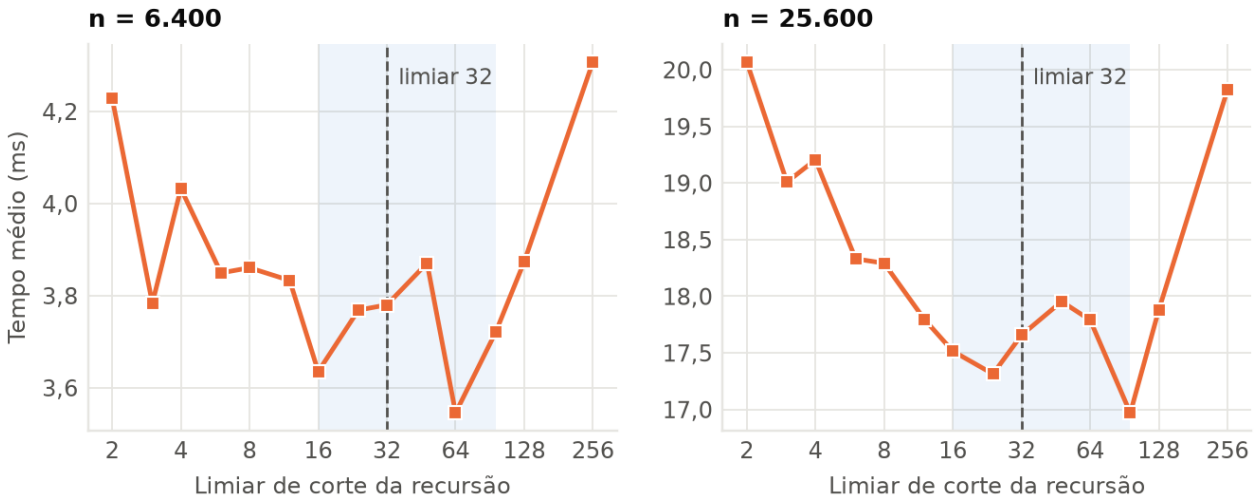


Figura 4. Efeito do limiar. A área destacada marca a faixa de valores praticamente empatados.

O resultado tem a forma de U prevista por Θ(n log(n/L) + nL). Limiares muito baixos fazem mais chamadas e alocações, enquanto limiares muito altos deixam folhas quadráticas grandes. Entre 16 e 96, porém, as diferenças ficam dentro do ruído, e o melhor valor exato mudou entre rodadas repetidas. Não existe, portanto, um único limiar ótimo nessas medições. O valor adotado foi **L = 32**, por estar no meio dessa faixa, longe das duas extremidades ruins.

A baixa sensibilidade tem uma explicação: as ordenações iniciais, que dominam o tempo total, não dependem do limiar. Note também que L = 32 é muito menor que a virada do algoritmo completo, entre 800 e 1.600 pontos: na folha, a força bruta compete só com a recursão, sem custo adicional de ordenação.

7. Conclusão

Os experimentos confirmaram as complexidades previstas: tempo quadrático nas forças brutas, crescimento próximo de n log n nas versões de divisão e conquista e linear nas versões O(n). Mesmo assim, a análise assintótica não conta a história inteira. As constantes definem quem vence em entradas pequenas, adiam a virada da Etapa 2 para perto de mil pontos e explicam por que uma versão O(n) pode empatar com outra Θ(n log n). A medição também exigiu cuidado: a coleta de lixo forçada chegou a distorcer os resultados.

Como limitação, as medições foram feitas com outros programas abertos, e os tempos muito curtos têm mais ruído. Além disso, os resultados são de uma única máquina, embora os CSVs brutos e as sementes fixas permitam repetir todo o experimento.